

Coding Challenge: Real-Time Notification System for a Gaming Platform

Globalli — Engineering Hiring Exercise

Introduction

Read the challenge carefully before you start writing code. Focus only on the required features described below; do not build beyond what is asked if you don't need to. We recommend you focus on the requirements and do additional polishing if you get extra time, but additional features are welcome if you decide to add them after you completed.

The use of AI tools is not just allowed, it is recommended. We want to see the process you followed to understand the requirements and build the features, not just the final files.

This is what we are expecting:

- The resulting product must compile and run. We will clone the repo and run it using the build scripts you provide.
- The delivery is done by uploading the project into a public GitHub repository you own.
- (Recommended) add a README.md file on the repo that explains how to run it and the main features you included.

The follow up interview will have two parts:

- **Your use of AI:** the prompts you used and the process you followed to go from requirements to a working solution. Workflows and patterns used, tools used. Keep that as part of the deliverable.
- **Defending the resulting product:** the architecture patterns you chose and why, the tradeoffs behind them, hypothetical scenarios on what will imply adding features on top of what you built, and general questions about how the system works.

Please feel free to reach out to the provided contacts of the HR representative if you have questions about the challenge.

Have fun with it.

Scenario

You are developing a real-time notification system for a popular online gaming platform. This platform hosts various multiplayer games, and players need to be kept informed about important events happening in the game world and their social network.

Specific Requirements

- **In-Game Events:** Players should receive real-time notifications about in-game events, such as:
 - Level Up: When a player reaches a new level.
 - Item Acquired: When a player obtains a rare or valuable item.
 - Challenge Completed: When a player completes a challenging quest or achievement.
 - Player vs. Player (PvP) Events: When a player is attacked or defeated by another player (optional, but good to consider).
- **Social Events:** Players should also receive notifications about social events:
 - Friend Request: When another player sends them a friend request.
 - Friend Accepted: When a friend request is accepted.
 - New Follower: When another player starts following them.
- **Multiple Notification Channels:** For this simplified coding exercise, focus on in-app notifications. Assume the game client has a mechanism to receive these notifications in real-time. (In a real-world scenario, you'd have push notifications, email, etc., but we're simplifying for the interview.)
- **Notification Content:** Notifications should be clear, concise, and informative. For example:
 - Level Up: "Congratulations! You've reached level 15!"
 - Item Acquired: "You've acquired the legendary Sword of Azeroth!"
 - Friend Request: "Player 'X' has sent you a friend request."
- **User Preferences (Simplified):** For this exercise, assume each user can enable or disable notifications for each category of events (e.g., "Game Events," "Social Events"). You can represent these preferences with a simple dictionary or map.

Technical Requirements

- **Notification Class:** You are provided with a `Notification` class.
- **Notification Handling:** Design and implement the necessary components to handle notifications. This will likely involve interfaces, services, and classes to manage the sending of notifications, checking user preferences, and interacting with the game and social systems. Consider how you will structure your code to ensure flexibility and maintainability as the system grows.
- **Event Handling:** Implement a mechanism to receive and process events from different parts of the game. This should include:
 - Receiving events: Assume that game events (like player leveling up or acquiring an item) and social events (like friend requests) are emitted by different parts of the game platform. You'll need a way to capture these events and pass them to your notification system. For this exercise, you can simulate these events using the provided examples below.
 - Dispatching notifications: Your notification system should determine the type of notification, create the `Notification` object, check user preferences, and send the notification.
- **Example Usage:** Demonstrate how different parts of the game (or simulated game events) would trigger notifications. The example usage should clearly show how events are triggered and how the notification system responds.

Example Event Triggers (for testing)

You can simulate these event triggers in your test code. Imagine these methods are called by various parts of the game platform:

```
gameEngine.playerLeveledUp(1, 15); // User 1 leveled up to 15
gameEngine.itemAcquired(2, "SwordOfAzeroth"); // User 2 acquired an item
socialSystem.friendRequestSent(3, 1); // User 3 sent a friend request to user 1
socialSystem.friendRequestAccepted(1, 3); // User 1 accepted friend request from 3
```

Important Note

The actual display of the notification in the game client is not part of this exercise's scope. You can assume that there is an existing notification client or mechanism to handle the final delivery and display of the notification. You are free to mock or simulate this as an external dependency. Focus your efforts on the core notification logic: receiving events, processing them, checking user preferences, and triggering the notification sending process.